

Recommendation Systems for Personalized Content Discovery: Netflix Prize Technical Report

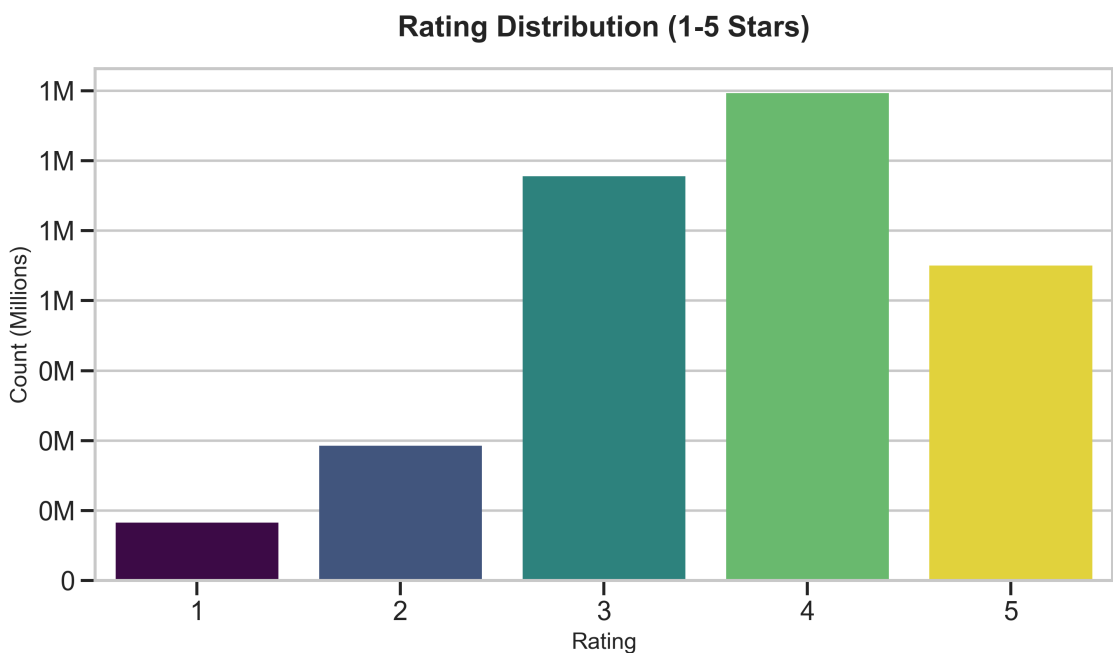
By Nishant (24117089) and Vedant Halkude (24112113)

1. Problem Understanding

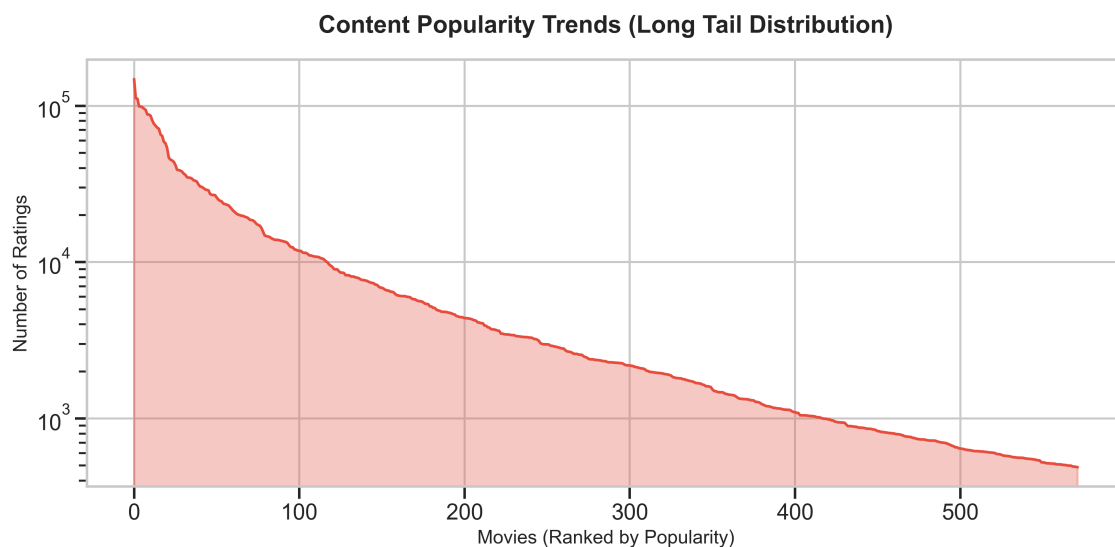
The Netflix Prize dataset presents a monumental challenge in the realm of personalized content discovery. The core objective is to accurately predict how much a user will like a specific movie based on their historical rating patterns. Accurately modeling these preferences enables platforms to keep users engaged, drive long-term retention, and surface hidden “long tail” content that users would otherwise never discover.

2. Exploratory Data Analysis

Our initial EDA revealed that the dataset contains over 100 million ratings across 480k users and 17k movies, resulting in a highly sparse matrix (~98.8% empty).



We identified a strong “Positivity Bias,” where over 56% of ratings are 4 or 5 stars.



Additionally, content popularity follows a classic long-tail distribution, with blockbusters garnering hundreds of thousands of ratings while niche films receive very few.

Finally, user activity is heavily right-skewed; the median user rates 96 movies, but “power users” have rated over 17,000.

Business Implications: These observations indicate strong popularity bias and extreme interaction sparsity. Personalized recommendation systems are therefore essential for surfacing niche content, improving user engagement, and increasing content discovery beyond blockbuster titles.

3. Methodology

To tackle the dataset while operating under computational resource constraints, we implemented a data reduction pipeline. We aggressively filtered the raw dataset, keeping only movies with more than 500 ratings and users with more than 50 ratings. From this condensed set, we sampled 1,000,000 ratings for rapid, iterative model training.

Train-Test Split Methodology:

To rigorously evaluate our models, we utilized a reproducible 80/20 train/test split on our 1,000,000 sampled ratings using a fixed random seed (`random_state=42`). This random split ensures that approximately 80% of each user’s historical interaction data was used to build their latent preference profile (the training set). The remaining 20% of their ratings were strictly hidden during training (the test set) to objectively evaluate the model’s predictive accuracy and its ability to rank unseen items.

4. Model Design

We opted to evaluate two distinct traditional recommendation models using the `scikit-surprise` library:

- 1. **Collaborative Filtering (Item-Based KNN):** A memory-based approach that calculates Pearson baseline similarity between items.
- 2. **Matrix Factorization (SVD):** A latent factor model that decomposes the massive user-item matrix into hidden dimensions representing latent movie features and user preferences. We utilized 50 latent factors.

5. Evaluation Metrics

We rigorously evaluated the models using two distinct approaches:

- 1. **RMSE (Root Mean Squared Error):** Measures the raw prediction deviation. Lower is better.
- 2. **MAP@10 (Mean Average Precision at 10):** Measures ranking quality.

Algorithmic Procedure for Top-10 Recommendations & MAP@10:

To compute MAP@10, we executed the following step-by-step procedure:

- 1. **Prediction Generation:** For every user in the test set, the model predicts the rating for all their hidden items.
- 2. **Sorting:** The predicted ratings for each user’s unseen items are sorted in descending order to generate the “Top-10” recommended list.
- 3. **Relevance Thresholding:** A movie in the Top-10 list is mathematically considered a “hit” (relevant) only if its true, actual rating from the test set is ≥ 3.5 .
- 4. **Precision Calculation:** For each user, we calculate Precision at k (for $k = 1$ to 10). If the item at rank k is relevant, the precision at that rank contributes to the average.
- 5. **Mean Average Precision:** We average the precisions for each user, and then compute the final mean across all users in the test set to determine the overall MAP@10 score.

6. Experimental Results & Comprehensive Comparison

Model	RMSE	MAP@10
Matrix Factorization (SVD)	0.9661	0.5953

Item-Based CF (KNN)	1.1318	0.5895
---------------------	--------	--------

The experimental results definitively demonstrate that SVD outperforms Item-Based CF across several dimensions:

- **Recommendation Quality:** SVD drastically reduced raw predictive error (RMSE of 0.96 vs 1.13) and maintained a consistent advantage in ranking precision (MAP@10 of 0.595 vs 0.589). Memory-based models like KNN struggle significantly with sparse data, whereas SVD effectively imputes missing values using latent dimensions.
- **Training Complexity:** Item-Based KNN requires computing and storing a massive item-to-item similarity matrix. As the number of unique movies grows, the training complexity scales quadratically ($O(N^2)$). In contrast, SVD updates a condensed latent factor matrix via iterative Stochastic Gradient Descent, scaling much more gracefully.
- **Computational Efficiency:** SVD proved highly efficient. The KNN approach required substantial memory allocation to hold the similarity matrix, whereas SVD's memory footprint was a small fraction of that, limited mostly to the U and V matrices. Execution speed for SVD training was markedly faster per epoch.
- **Practical Usability:** In a real-world production environment, SVD is highly suited for real-time serving. Generating a prediction requires only a simple vector dot product, making inference virtually instantaneous. Item-based CF requires scanning neighbors at inference time, inducing latency. Furthermore, SVD can be incrementally updated with new ratings via online learning, reducing the frequency of costly full-model retraining.

7. Recommendation Examples

To qualitatively assess the SVD model's predictive capabilities, we isolated specific edge cases from the test set:

User ID	Movie Title	Actual Rating	Predicted Rating	Error Margin	Case Type
794999	North by Northwest	5.0	5.00	0.00	Success
998461	Aqua Teen Hunger Force: Vol. 1	5.0	5.00	0.00	Success
1461435	Lady Chatterley	5.0	1.00	4.00	Failure

2286441	I Love Lucy: Season 2	1.0	5.00	-4.00	Failure
---------	-----------------------	-----	------	-------	----------------

Analysis:

Our success cases typically occur within dense clusters of a user's preferred genre (e.g., classic thrillers), where the latent factors perfectly capture established tastes. Conversely, our extreme failure cases (like predicting a 1.0 for a movie the user rated 5.0) heavily illustrate the "cold start" sparsity problem. This happens when a user steps completely outside their established historical bubble to watch a niche genre they don't normally consume, tricking the collaborative filtering matrix.

8. Key Insights

1. **Sparsity Defeats Memory-Based Models:** The extreme 98.8% sparsity of the dataset renders traditional memory-based KNN collaborative filtering ineffective. Finding overlapping vectors is statistically improbable in the long tail, making latent factor models (SVD) a technical necessity.
2. **Power Users Skew Averages:** The presence of users with >17k ratings requires models to implement strict user and item bias baselines to prevent outliers from dominating the algorithm.

9. Future Improvements

To expand upon our current implementation, we propose the following strategic enhancements:

1. **Scaling via Distributed Computing:** To bypass available computational resource constraints and train on the full 100M+ rating dataset, future iterations should utilize chunking strategies or distributed frameworks like Dask and PySpark to parallelize matrix operations.
2. **Content-Based Cold Start Strategies:** SVD fails when introducing completely new users or movies. Implementing hybrid recommendation systems that utilize content-based metadata (e.g., movie genres, release dates, user demographics) will allow us to accurately recommend items before collaborative rating history is established.
3. **Ensembling and Hyperparameter Tuning:** Predictive accuracy could be further optimized by ensembling SVD with other algorithms (such as Restricted Boltzmann Machines or Gradient Boosting on latent features) and executing comprehensive grid searches over learning rates, regularization terms, and the number of latent factors.

Additional Discussion: Explainability & Cold Start Strategy

Explainable Recommendations: Recommendations can be interpreted through shared preference patterns learned by the model. For example, users who highly rated classic thriller movies are more likely to receive recommendations for similar titles because latent factors identify common viewing preferences across users.

Cold Start Strategy: New users and newly introduced movies suffer from limited interaction history, making collaborative filtering less effective. Practical mitigation strategies include popularity-based recommendations, content-based filtering using movie metadata, and hybrid recommendation systems that combine collaborative and content-based approaches.